@Mock as Method/Constructor Parameter

In previous topic, we saw usage of @InjectMocks and how it works.

```java
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    PaymentGateway gateway;

    @Mock
    OrderRepository repo;

    @InjectMocks
     OrderService service;

    @Test
    void placeOrder_success() {
        Mockito.when(gateway.charge(100)).thenReturn(true);
        boolean result = service.placeOrder(100);
        assertTrue(result);
        Mockito.verify(repo).save();
    }
}

```

In Mockito Extension

```java
public class MockitoExtension implements BeforeEachCallback, AfterEachCallback, ParameterResolver {
```

And in this, we have seen how, ***MockitoExtension*** add hook in our code, and:

@BeforeEach: in before each method, resolves @Mock and @InjectMock.

Concept && Coding

@AfterEach: in after each method, close the resources.

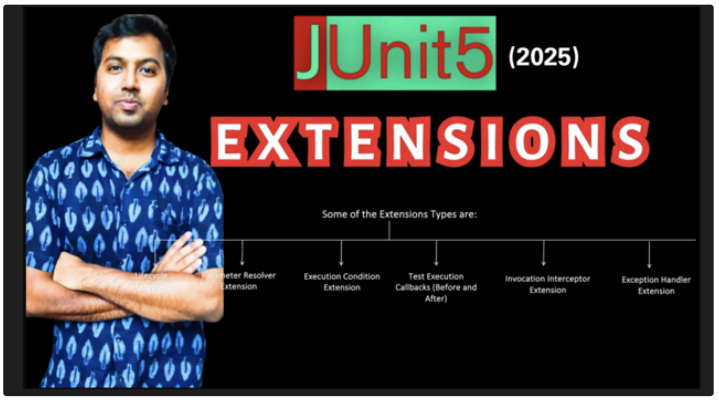
Now, as a curious engineer, one thing might comes to your mind

In "***MockitoExtension***" I understand BeforeEachCallback and AfterEachCallback use.

But Why "***MockitoExtension***" also implements "ParameterResolver"

```java
public class MockitoExtension implements BeforeEachCallback, AfterEachCallback, ParameterResolver {
```

Again, ParameterResolver also I have covered in "Extensions" topic



Recall from "Extensions" topic, how Parameter Resolver works.

```
@ExtendWith(StudentParameterResolver.class)
class MyServiceTest {

    @Test
    void testMethod1(Student student) {
        System.out.println("Student name is: " + student.name);
    }

}

```

- Junit, try to invoke "testMethod1" but it require "Student" object.

So to resolve this:

- It will call each Extension which implements "ParameterResolver".
- And check with them, if they can create the Student object.

```
public class StudentParameterResolver implements ParameterResolver {

    @Override
    public boolean supportsParameter(ParameterContext parameterContext,
                                     ExtensionContext extensionContext) throws ParameterResolutionException {

        return parameterContext.getParameter().getType() == Student.class;
    }

    @Override
    public Object resolveParameter(ParameterContext parameterContext,
                                   ExtensionContext extensionContext) throws ParameterResolutionException {

        return new Student(name: "myName", age: 27);
    }
}

```

Concept && Coding

Similarly, "***MockitoExtension***" implements ParameterResolver

- And it support and can resolve @Mock annotations.

MockitoExtension framework class:

```
@Override
public boolean supportsParameter(ParameterContext parameterContext, ExtensionContext context)
        throws ParameterResolutionException {

    return parameterResolver.supportsParameter(parameterContext, context);
}

@Override
@unchecked/
public Object resolveParameter(ParameterContext parameterContext, ExtensionContext context)
        throws ParameterResolutionException {

    Object resolvedParameter = parameterResolver.resolveParameter(parameterContext, context);
    if (resolvedParameter instanceof ScopedMock) {
        context.getStore(MOCKITO).get(MOCKS, Set.class).add(resolvedParameter);
    }

    return resolvedParameter;
}

```

So yes, we can use @Mock annotations as parameter too:

Used as Parameter In Constructor:

```java
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    private final PaymentGateway gateway;
    private final OrderRepository repo;
    private final OrderService service;
  /*
    Benefit:
    - Clean
    - We can make fields 'final'.


    Cons:
    - Risky for PER_CLASS lifecycle. As constructor is invoked
      only when object is created.

    - @InjectMock can not be used. As it can only resolve @Mock objects.
  */

    OrderServiceTest(@Mock PaymentGateway gateway, @Mock OrderRepository repo) {

        this.gateway = gateway;
        this.repo = repo;
        this.service = new OrderService(gateway, repo);
    }

    @Test
    void placeOrder_success() {
        when(gateway.charge(100)).thenReturn(true);

        boolean result = service.placeOrder(100);

        assertTrue(result);
        verify(repo).save();
    }
}

```

Concept && Coding

Used as Parameter In Method:

```java
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {


    @Test
  /*
  Mocks are created only for this method
  */
    void placeOrder_success(@Mock PaymentGateway gateway, @Mock OrderRepository repo) {

       OrderService service = new OrderService(gateway, repo);

      when(gateway.charge(100)).thenReturn(true);

        boolean result = service.placeOrder(100);

        assertTrue(result);
        verify(repo).save();
    }
}

```